Experiment - 1.1.1. Area of Circle

1. Aim

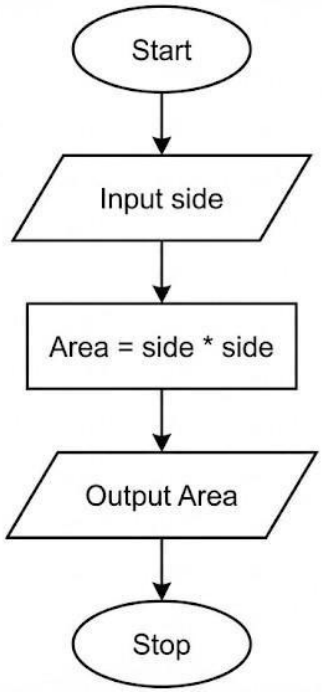
To write a Python program that calculates the area of a circle given its radius as input from

the user, using the constant value of $\pi = 3.14$ and displaying the result formafled to four decimal places.

2. Pseudocode

1. **START**
2. **READ** the input value from the user and convert it to a floating-point number.
3. **STORE** the value in a variable named radius.
4. **DEFINE** a constant variable pi and assign it the value 3.14.
5. **CALCULATE** the area using the formula: $Area = \pi \times radius \times radius$.
6. **FORMAT** the resulting area to 4 decimal places.
7. **PRINT** the formafled area.
8. **END**

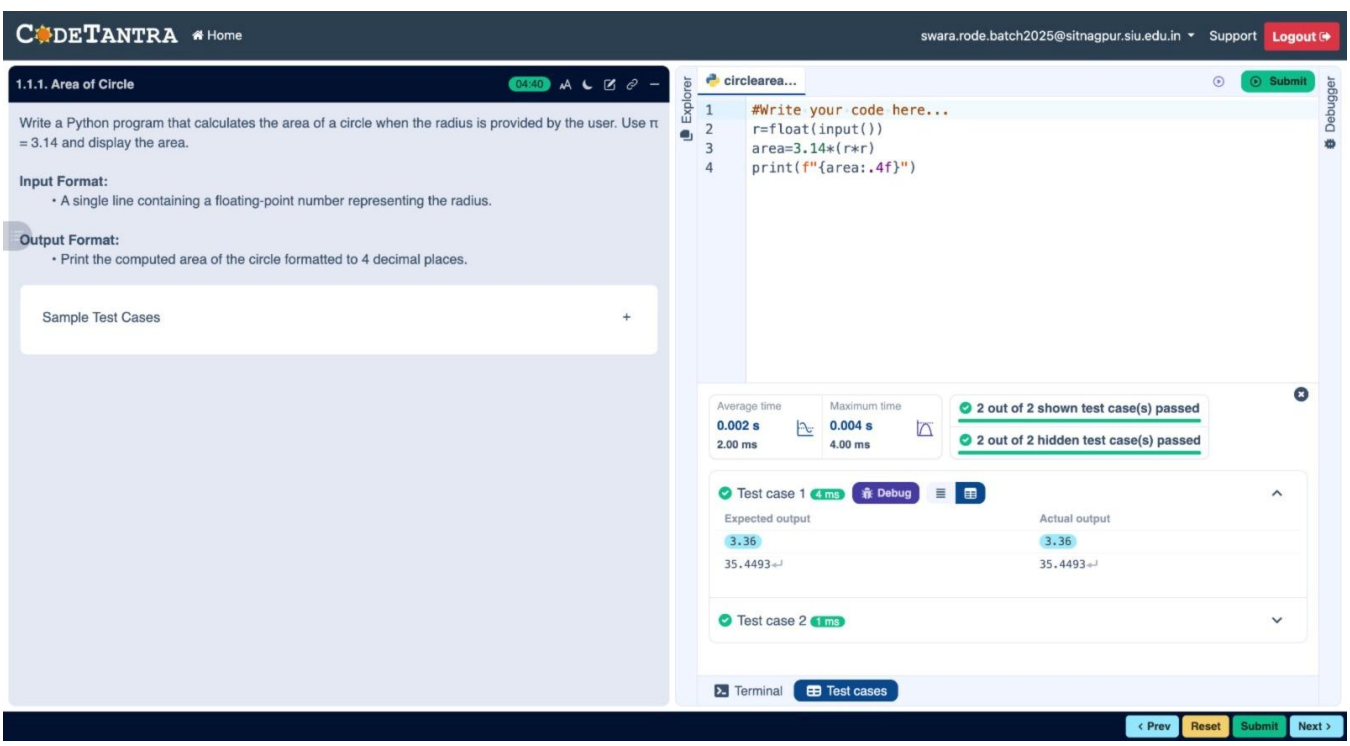
3. Flowchart



4. Python Program

```
# Program to calculate the area of a circle
# Input: Radius as a float
# Output: Area formafled to 4 decimal places

# Taking input from the user
radius = float(input())

# Constant value of Pi
pi = 3.14

# Calculating area
area = pi * radius * radius

# Displaying the result formafled to 4 decimal places
print(f"{area:.4f}")
```

5. Experiment Screenshot



Experiment - 1.1.2. Area ofi Rectangle

1. Aim

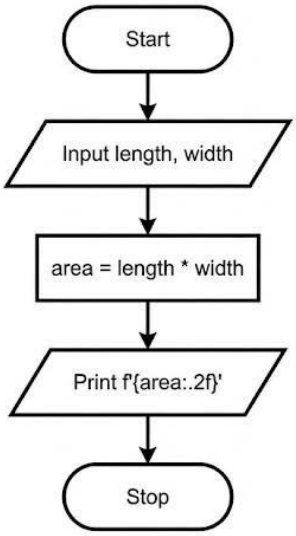
To design and implement a Python program that calculates the area of a rectangle. The program accepts the length and width as floating-point inputs from the user and computes

the area using the formula $Area = Length \times Width$, displaying the result formafled to two decimal places.

2. Pseudocode

1. **START**
2. **READ** the first input value and store it as a float in the variable length.
3. **READ** the second input value and store it as a float in the variable width.
4. **CALCULATE** the area by multiplying length and width.
5. **STORE** the result in the variable area.
6. **FORMAT** the value of area to 2 decimal places.
7. **PRINT** the formafled result.
8. **END**

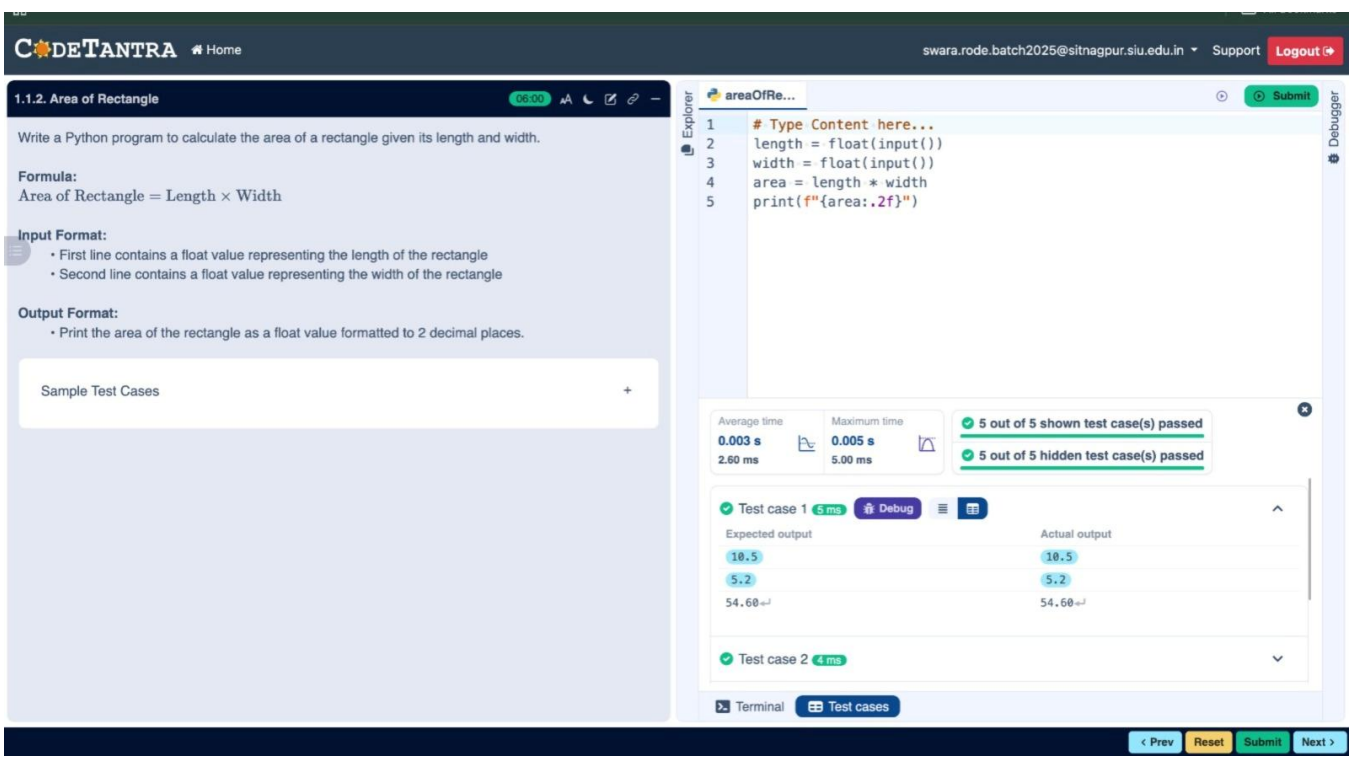
3. Flowchart



4. Python Program

```
# Program to calculate the area of a rectangle
# Input: Length and Width as floats
# Output: Area formafled to 2 decimal places

# Taking inputs from the user
length = float(input())
width = float(input())

# Calculating area
area = length * width

# Displaying the result formafled to 2 decimal places
print(f"{area:.2f}")
```

5. Experiment Screenshot



Experiment - 1.1.3. Calculate Area ofi the Square

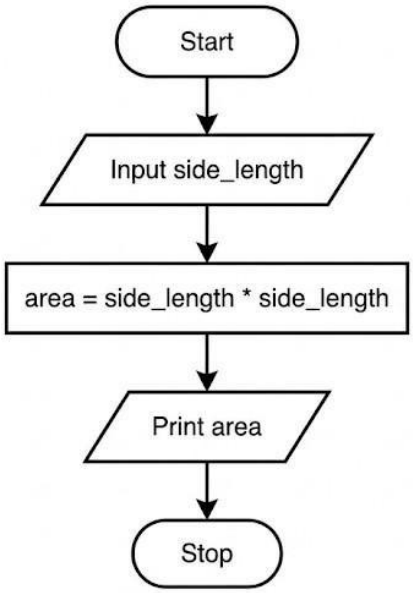
1. Aim

To design and implement a Python program that calculates the area of a square. The program accepts the side_length as an integer input and computes the area using the formula $Area = side_length^2$.

2. Pseudocode

1. **START**
2. **READ** the input value from the user and convert it to an integer.
3. **STORE** this value in the variable side_length.
4. **CALCULATE** the area by multiplying side_length by itself (side_length * side_length).
5. **STORE** the result in the variable area.
6. **PRINT** the value of area.
7. **END**

3. Flowchart

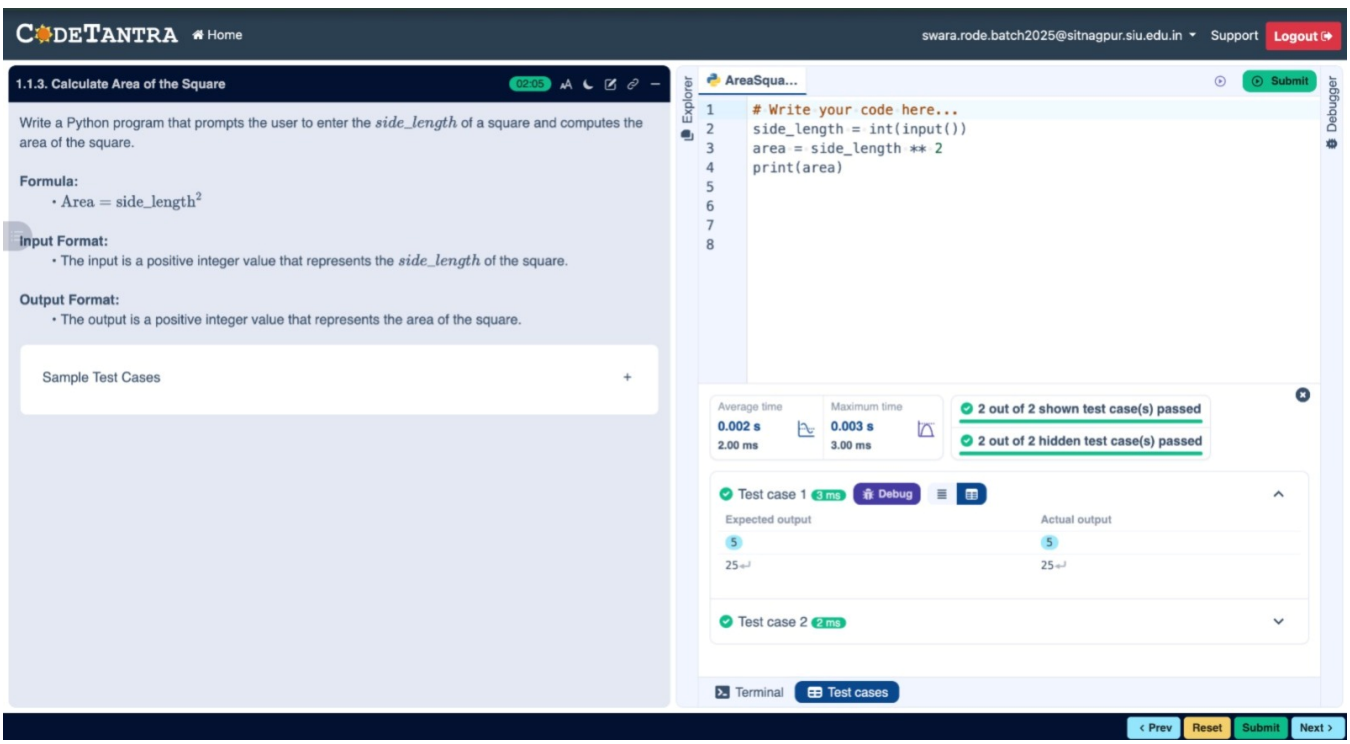


4. Python Program

```
# Program to calculate the area of a square
# Input: side_length as an integer
```

```
# Output: area as an integer

# Taking input from the user
side_length = int(input())

# Calculating area (side * side)
area = side_length * side_length

# Displaying the result
print(area)
```

5. Experiment Screenshot



Experiment - 1.1.4. Area ofi Triangle

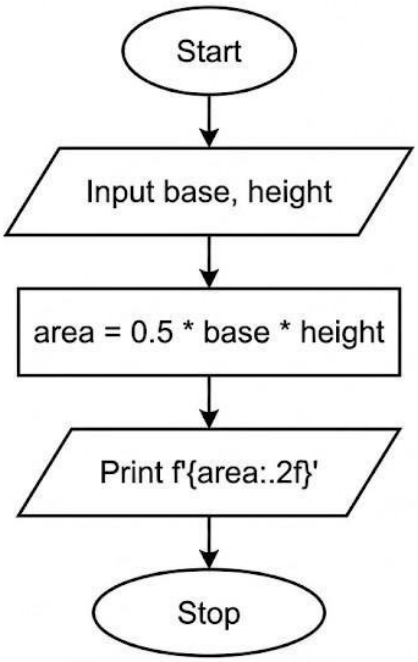
1. Aim

To design and implement a Python program that calculates the area of a triangle. The program prompts the user to enter the base and height as floating-point values and computes the area using the formula $Area = 0.5 \times base \times height$, with the result formafled to two decimal places.

2. Pseudocode

1. **START**
2. **READ** the first input value and store it as a float in the variable base.
3. **READ** the second input value and store it as a float in the variable height.
4. **CALCULATE** the area using the formula: 0.5 * base * height.
5. **STORE** the result in the variable area.
6. **FORMAT** the result to show 2 decimal places.
7. **PRINT** the formafled area.
8. **END**

3. Flowchart

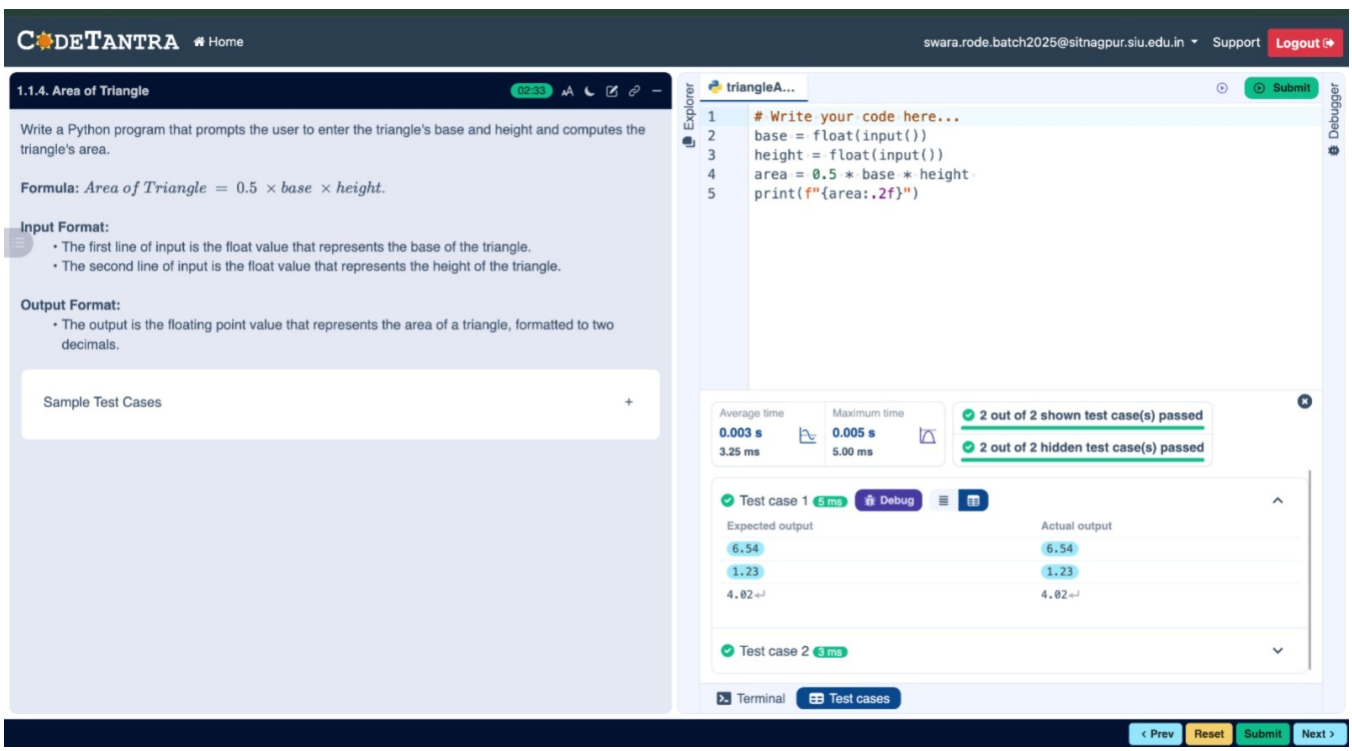


4. Python Program

Program to calculate the area of a triangle

```
# Input: Base and Height as float values
# Output: Area formafled to two decimal places

# Taking inputs from user
base = float(input())
height = float(input())

# Calculating the area
area = 0.5 * base * height

# Displaying the output formafled to 2 decimal places
print(f"{area:.2f}")
```

5. Experiment Screenshot



Experiment - 1.1.5. Student Pass or Fail Status

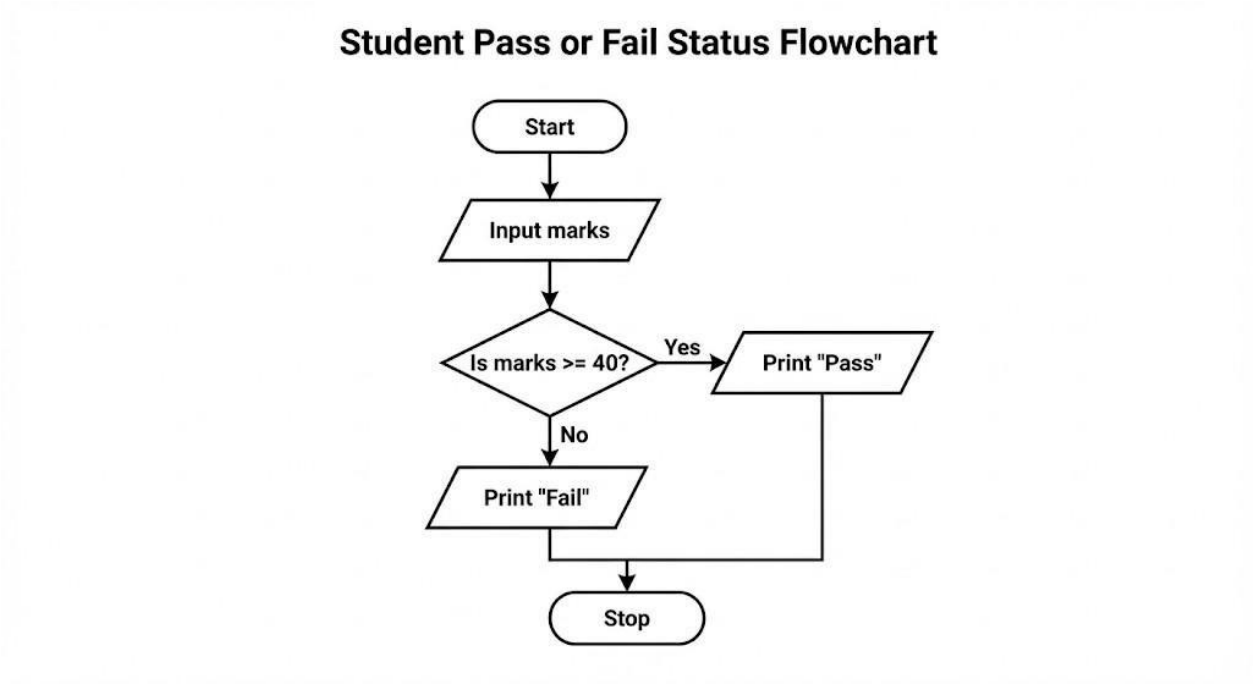
1. Aim

To design and implement a Python program that determines a student's examination status based on their marks. The program evaluates whether the student has passed or failed using a threshold of 40 marks (Pass: $marks \geq 40$, Fail: $marks < 40$).

2. Pseudocode

1. **START**
2. **READ** the input value from the user and convert it to an integer.
3. **STORE** the value in the variable marks.
4. **IF** marks is greater than or equal to 40:
    - **PRINT** "Pass"
5. **ELSE**:
    - **PRINT** "Fail"
6. **END**

3. Flowchart

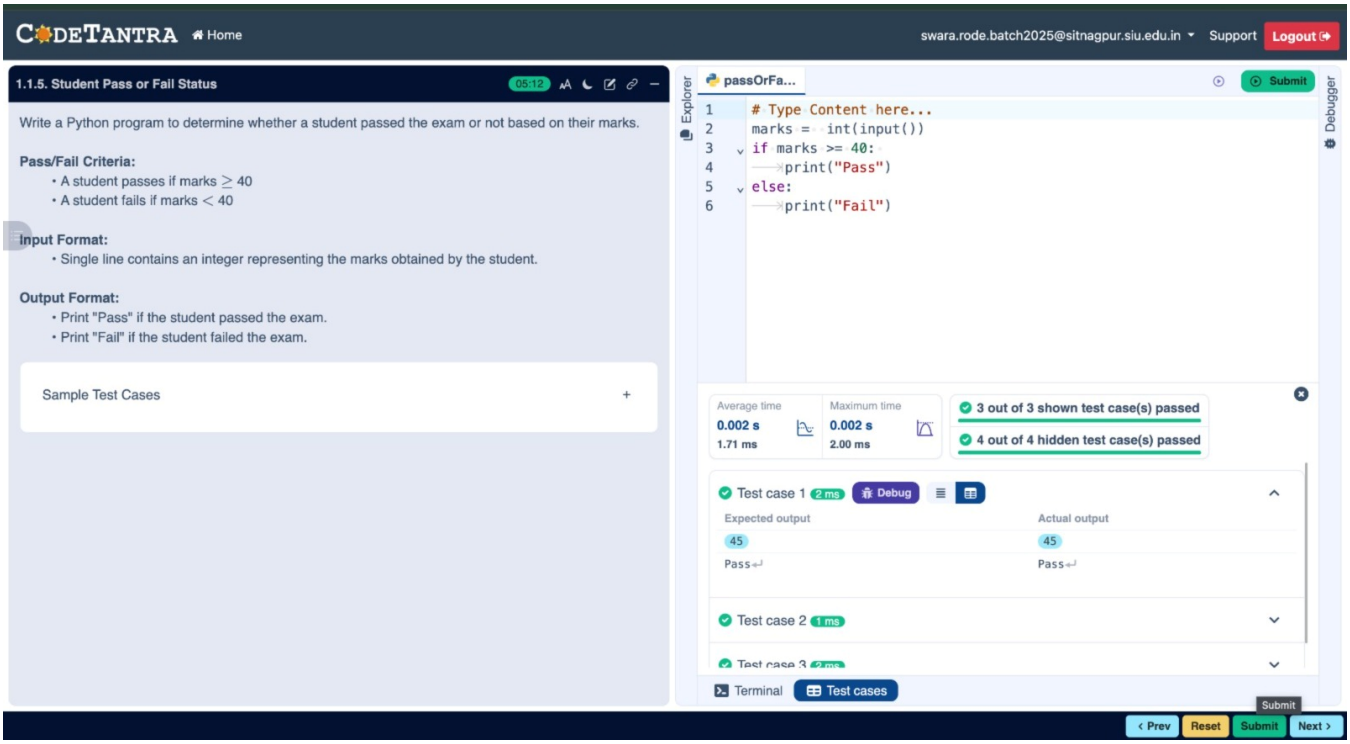


4. Python Program

```
# Program to determine student pass/fail status
# Input: marks as an integer
```

```
# Output: "Pass" or "Fail" based on criteria

# Taking marks as input
marks = int(input())

# Conditional check for passing criteria
if marks >= 40:
    print("Pass")
else:
    print("Fail")
```

5. Experiment Screenshot



Experiment - 2.1.1. Roots of a Quadratic Equation

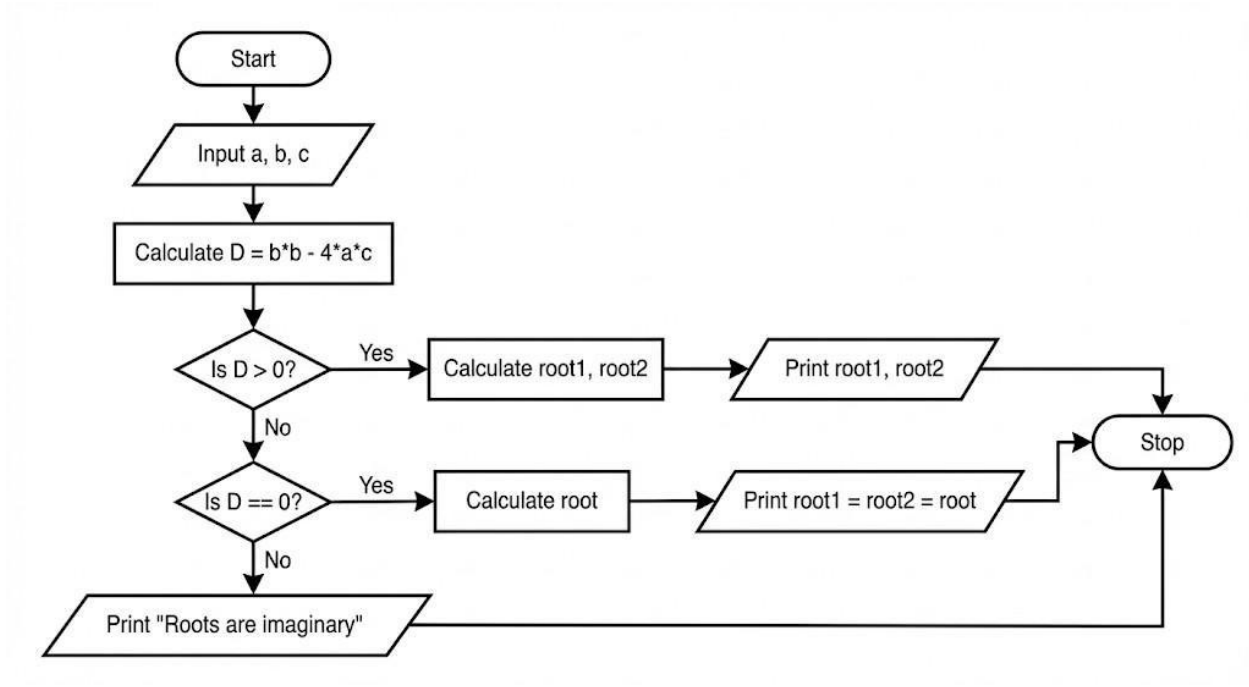
1. Aim

To design and implement a Python program that finds the roots of a quadratic equation $ax^2 + bx + c = 0$ using the quadratic formula. The program determines the nature of the roots (real and distinct, real and equal, or imaginary) based on the discriminant and formats $D = b^2 - 4ac$ the results to two decimal places.

2. Pseudocode

1. **START**
2. **READ** three space-separated integers a, b, and c.
3. **CALCULATE** the discriminant: $D = b^2 - 4ac$.
4. **IF** $D > 0$:
   - Calculate $root1 = \frac{-b+\sqrt{D}}{2a}$
   - Calculate $root2 = \frac{-b-\sqrt{D}}{2a}$
   - **PRINT** "root1 = [value]" and "root2 = [value]" formafled to 2 decimal places.
5. **ELSE IF** $D == 0$:
   - Calculate $root = \frac{-b}{2a}$
   - **PRINT** "root1 = root2 = [value]" formafled to 2 decimal places.
6. **ELSE** (if $D < 0$):
   - **PRINT** "Roots are imaginary".
7. **END**

3. Flowchart

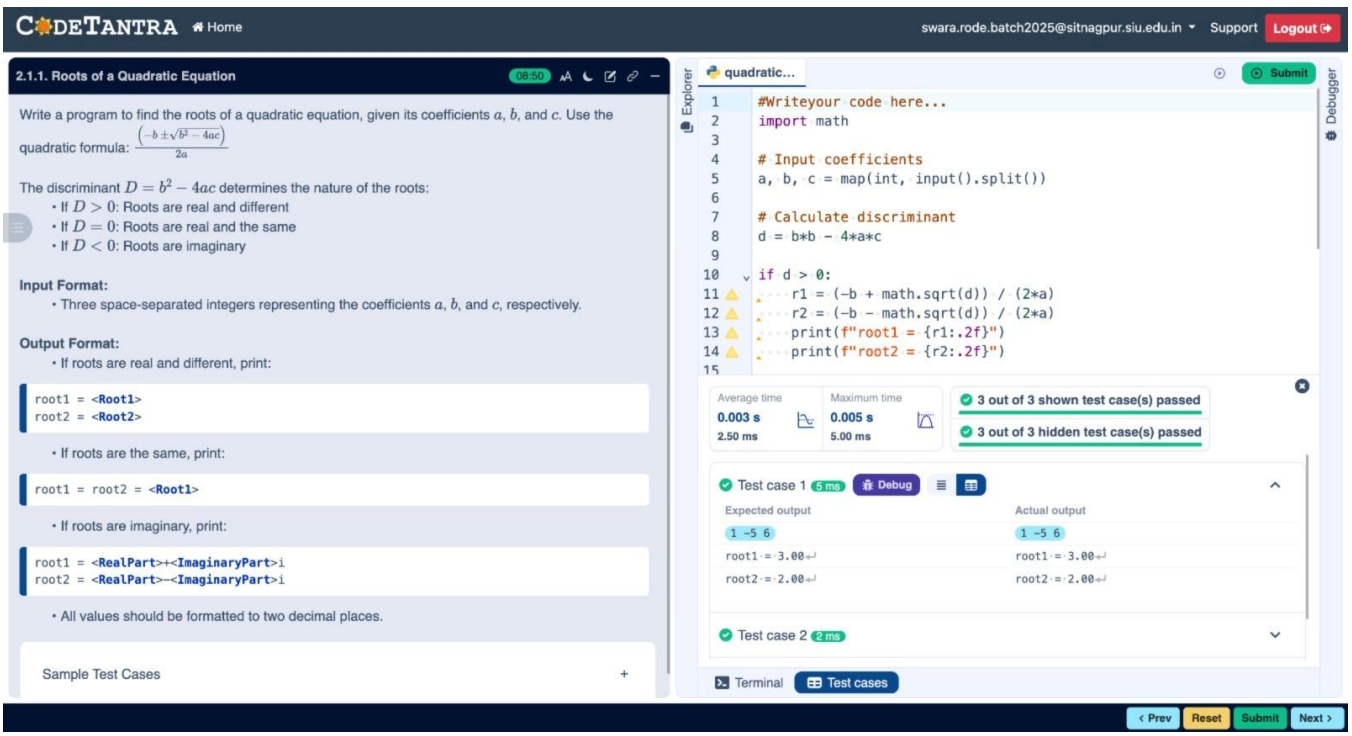


4. Python Program

```
# Program to find the roots of a quadratic equation
# Input: Space-separated coefficients a, b, and c
# Output: Roots formafled to 2 decimal places or nature of roots

import math

# Taking coefficients as input
a, b, c = map(int, input().split())

# Calculating the discriminant
D = b*b - 4*a*c

if D > 0:
    # Real and different roots
    sqrt_D = D ** 0.5
    root1 = (-b + sqrt_D) / (2 * a)
    root2 = (-b - sqrt_D) / (2 * a)
    print(f"root1 = {root1:.2f}")
    print(f"root2 = {root2:.2f}")

elif D == 0:
```

```
    # Real and same roots
    root = -b / (2 * a)
    print(f"root1 = root2 = {root:.2f}")

else:
    # Imaginary roots
    print("Roots are imaginary")
```

5. Experiment Screenshot



Experiment - 3.1.1. Largest ofi Three Numbers

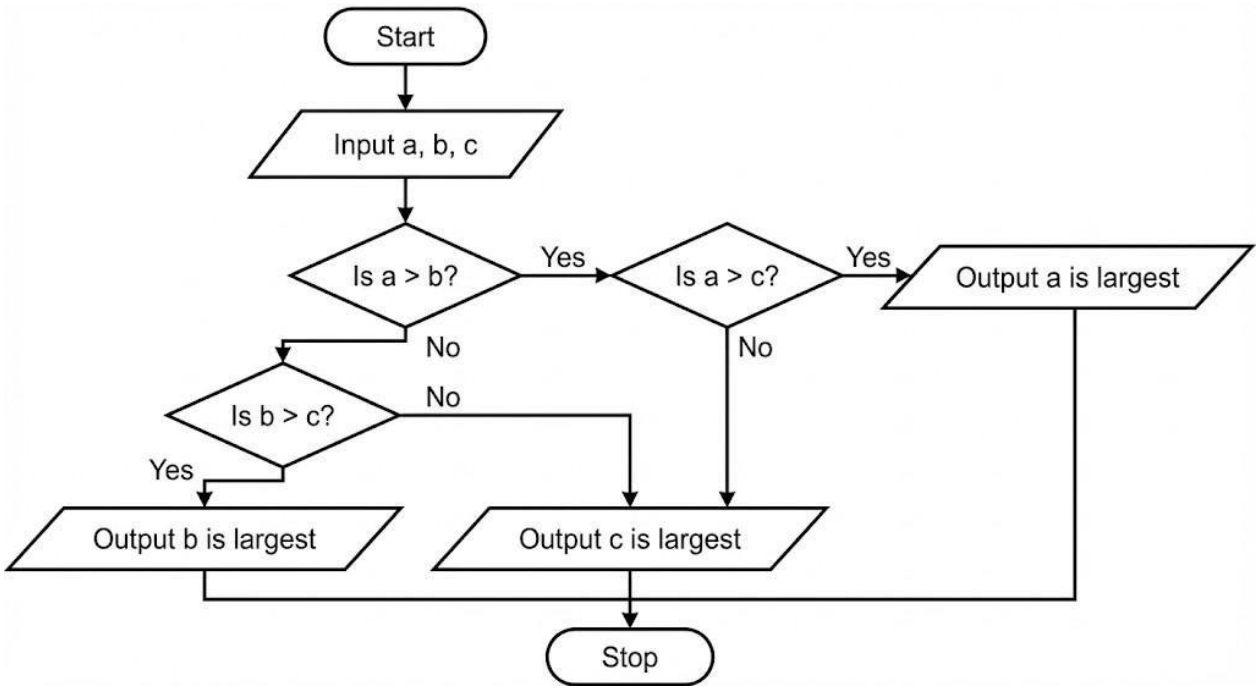
1. Aim

To design and implement a Python program that prompts the user to enter three integers and determines the largest among them using conditional statements (if-elif-else).

2. Pseudocode

1. **START**
2. **READ** three integers a, b, and c from the user (one per line).
3. **INITIALIZE** a variable largest to 0.
4. **IF** a >= b AND a >= c:
   - Set largest = a.
5. **ELSE IF** b >= a AND b >= c:
   - Set largest = b.
6. **ELSE**:
   - Set largest = c.
7. **PRINT** the value of largest.
8. **END**

3. Flowchart

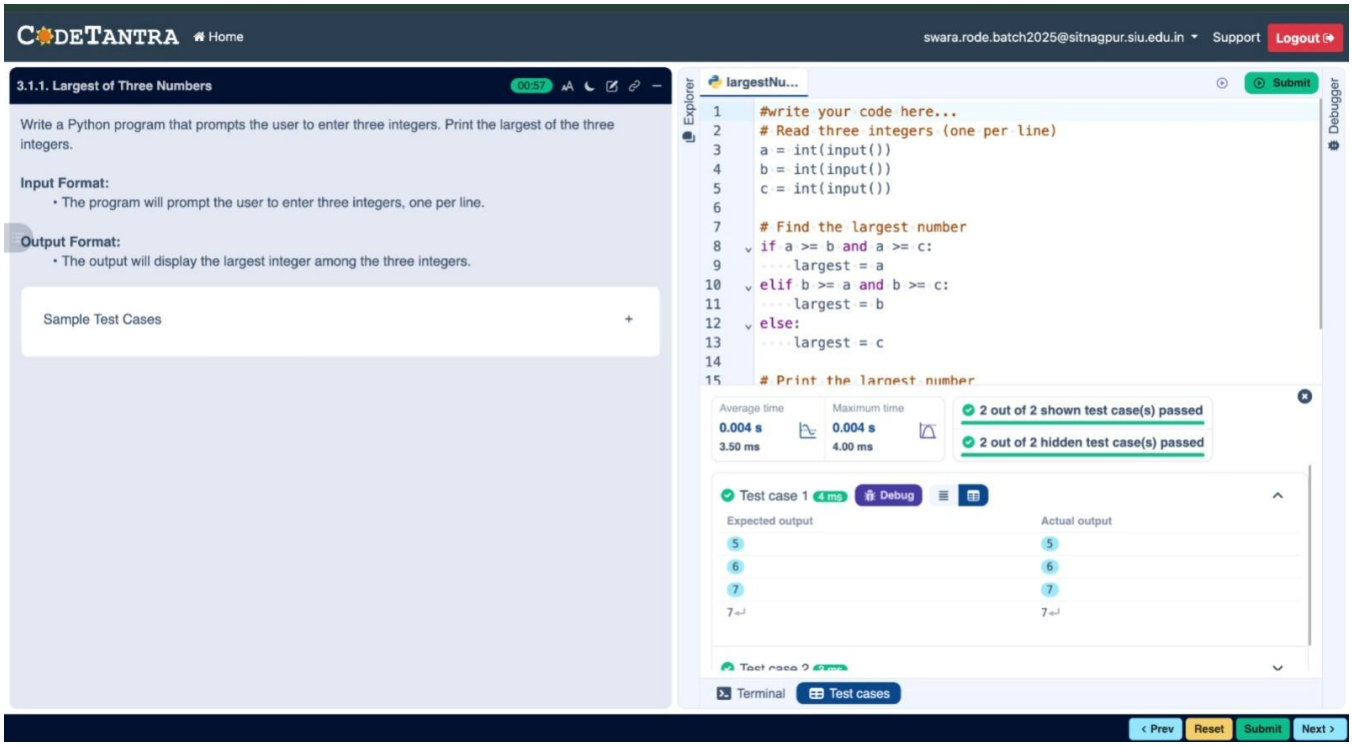


4. Python Program

Program to find the largest of three integers

```
# Input: Three integers, one per line
# Output: The largest integer among the three

# Taking three integers as input
a = int(input(""))
b = int(input(""))
c = int(input(""))

# Initializing variable to store the largest value
largest = 0

# Comparative logic using logical 'and'
if (a >= b) and (a >= c):
    largest = a
elif (b >= a) and (b >= c):
    largest = b
else:
    largest = c

# Displaying the result
print(largest)
```

5. Experiment Screenshot



Experiment - 3.1.2. Celsius to Fahrenheit

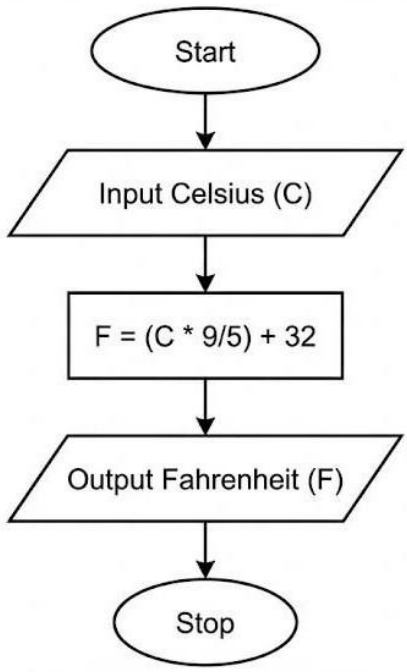
1. Aim

To design and implement a Python program that converts temperature from Celsius to Fahrenheit. The program accepts a floating-point value representing Celsius and applies the formula $Fahrenheit = (Celsius \times \frac{9}{5}) + 32$, displaying the result formafled to two decimal places.

2. Pseudocode

1. **START**
2. **READ** the input value as a float and store it in variable a (Celsius).
3. **CALCULATE** the Fahrenheit value using the formula: b = a * (9/5) + 32.
4. **STORE** the result in variable b.
5. **FORMAT** the value of b to 2 decimal places.
6. **PRINT** the formafled result.
7. **END**

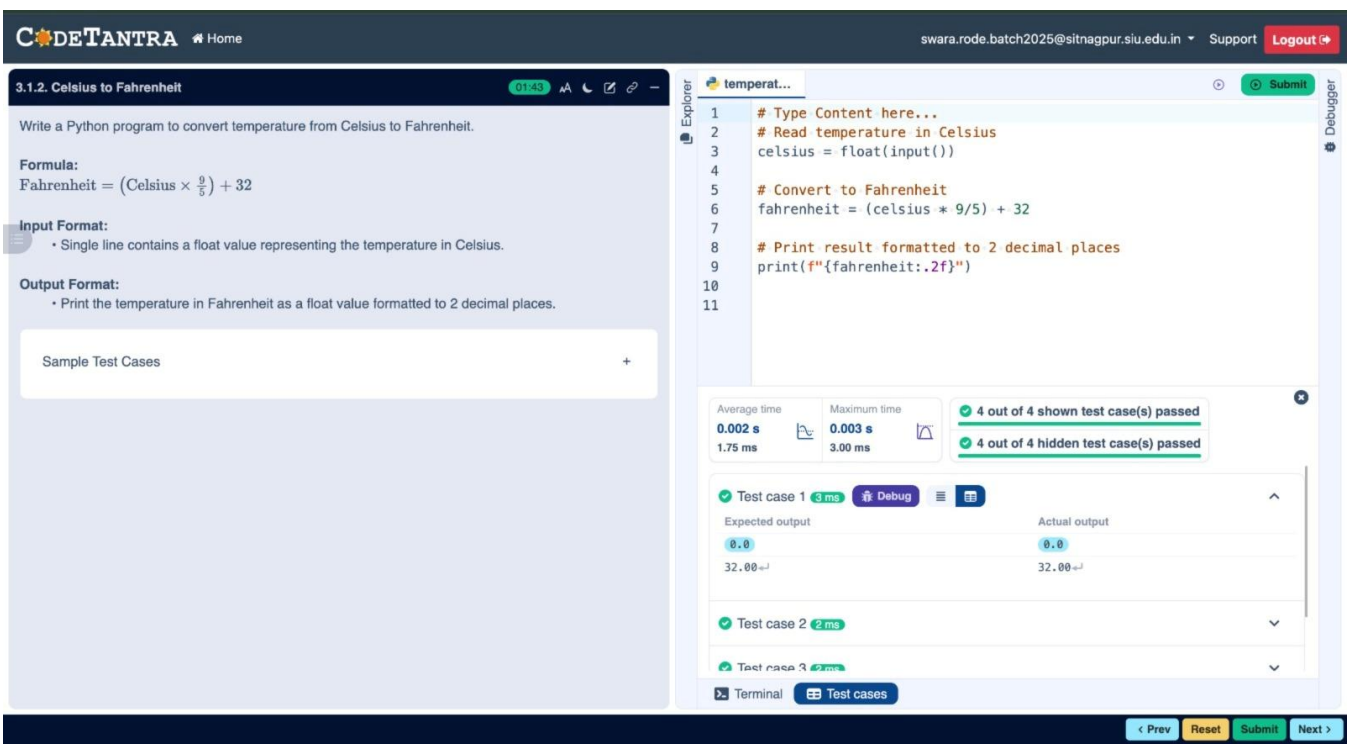
3. Flowchart



4. Python Program

```
# Program to convert Celsius to Fahrenheit
# Input: Celsius value as a float
# Output: Fahrenheit value formafled to 2 decimal places

# Taking Celsius input from the user
a = float(input(""))

# Applying the conversion formula
b = a * (9/5) + 32

# Displaying the result formafled to 2 decimal places
print(f"{b:.2f}")
```

5. Experiment Screenshot



Experiment - 5.1.2. Student Grade Based on Aggregate

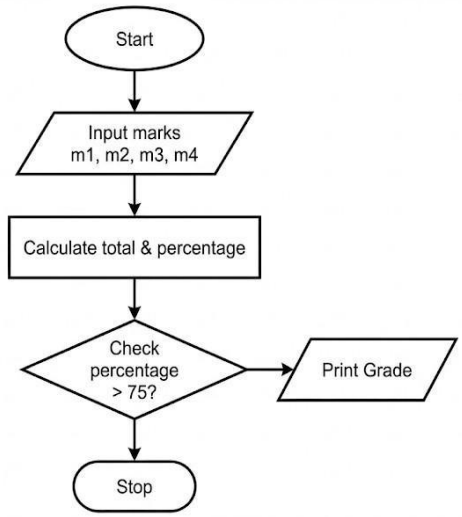
1. Aim

To design and implement a Python program that calculates a student's total marks, aggregate percentage, and final grade based on the marks obtained in fiour subjects. The program determines the grade using a specific set ofi percentage-based conditions.

2. Pseudocode

1. **START**
2. **READ** fiour space-separated integers firom the user and **STORE** them in variables m1, m2, m3, and m4.
3. **CALCULATE** the sum ofi the fiour subjects and **STORE** it in the variable total (total = m1 + m2 + m3 + m4).
4. **CALCULATE** the aggregate percentage by dividing the total by 4 and **STORE** it in the variable percentage.
5. **CHECK** the value ofi percentage to determine the grade:
    - **IF** percentage > 75, **STORE** "Distinction" in grade.
    - **ELSE IF** percentage >= 60, **STORE** "First Division" in grade.
    - **ELSE IF** percentage >= 50, **STORE** "Second Division" in grade.
    - **ELSE IF** percentage >= 40, **STORE** "Third Division" in grade.
    - **ELSE**, **STORE** "Fail" in grade.
6. **PRINT** the total marks.
7. **PRINT** the percentage fiormatted to two decimal places.
8. **PRINT** the grade.
9. **END**

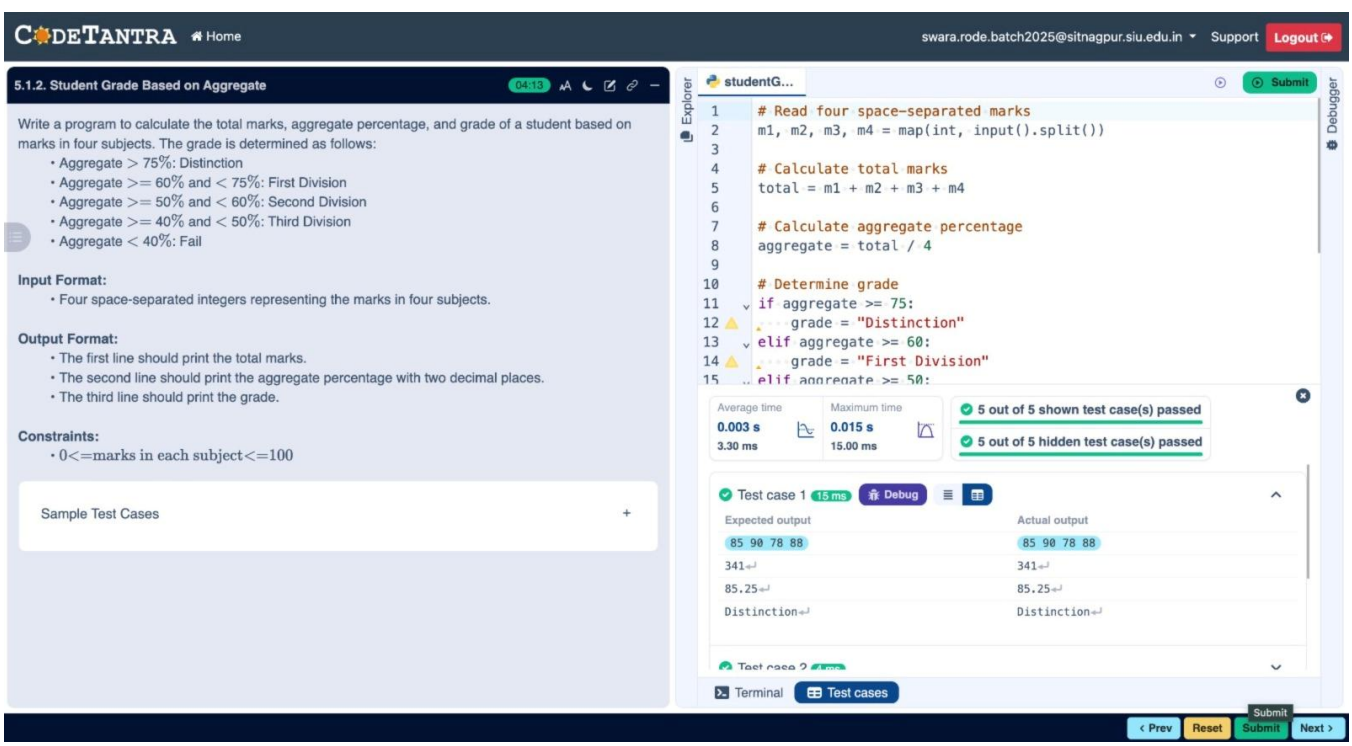
3. Flowchart



4. Python Program

```
# Read fiour subject marks
m1, m2, m3, m4 = map(int, input().split())

# Calculate total and percentage
total = m1 + m2 + m3 + m4
percentage = total / 4

# Determine the grade
ifi percentage > 75:
   grade = "Distinction"
elifi percentage >= 60:
   grade = "First Division"
elifi percentage >= 50:
   grade = "Second Division"
elifi percentage >= 40:
   grade = "Third Division"
else:
   grade = "Fail"

# Output the results
print(total)
print(fi"{percentage:.2fi}")
print(grade)
```

5. Experiment Screenshot



Experiment - 6.1.1. Incremented Date

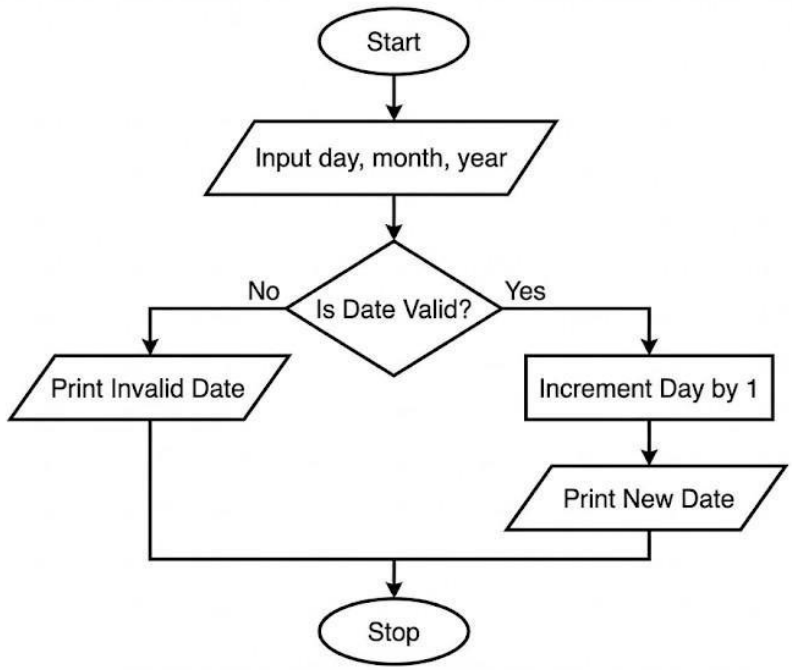
1. Aim

To design and implement a Python program that checks ifi a given date (day, month, year) is valid according to the Gregorian calendar rules. Ifi the date is valid, the program computes and prints the next day's date (incremented date) in DD-MM-YYYY fiormat. Ifi the date is invalid, it prints "Invalid Date".

2. Pseudocode

1. **START**
2. **READ** three integers firom the user representing the day, month, and year, and **STORE** them in variables d, m, and y.
3. **INITIALIZE** a boolean variable flag to True to keep track ofi date validity.
4. **CHECK** basic validity: **IF** y <= 0 **OR** m < 1 **OR** m > 12, **SET** flag to False.
5. **IF** flag is True:
    - **DETERMINE** maxdays based on the month:
        - **IF** m is 1, 3, 5, 7, 8, 10, or 12: maxdays = 31.
        - **ELSE IF** m is 4, 6, 9, or 11: maxdays = 30.
        - **ELSE IF** m is 2 (February):
            - **IF** the year y is a leap year ((y % 400 == 0) or (y % 4 == 0 and y % 100 != 0)): maxdays = 29.
            - **ELSE**: maxdays = 28.
    - **CHECK** day validity: **IF** d < 1 **OR** d > maxdays, **SET** flag to False.
6. **IF** flag is True (The date is completely valid):
    - **INCREMENT** the day d by 1.
    - **IF** d > maxdays (End ofi the month reached):
        - **SET** d = 1.
        - **INCREMENT** the month m by 1.
        - **IF** m > 12 (End ofi the year reached):
            - **SET** m = 1.
            - **INCREMENT** the year y by 1.
    - **PRINT** the incremented date fiormatted as DD-MM-YYYY (zero-padded).
7. **ELSE**:
    - **PRINT** "Invalid Date".
8. **END**

3. Flowchart

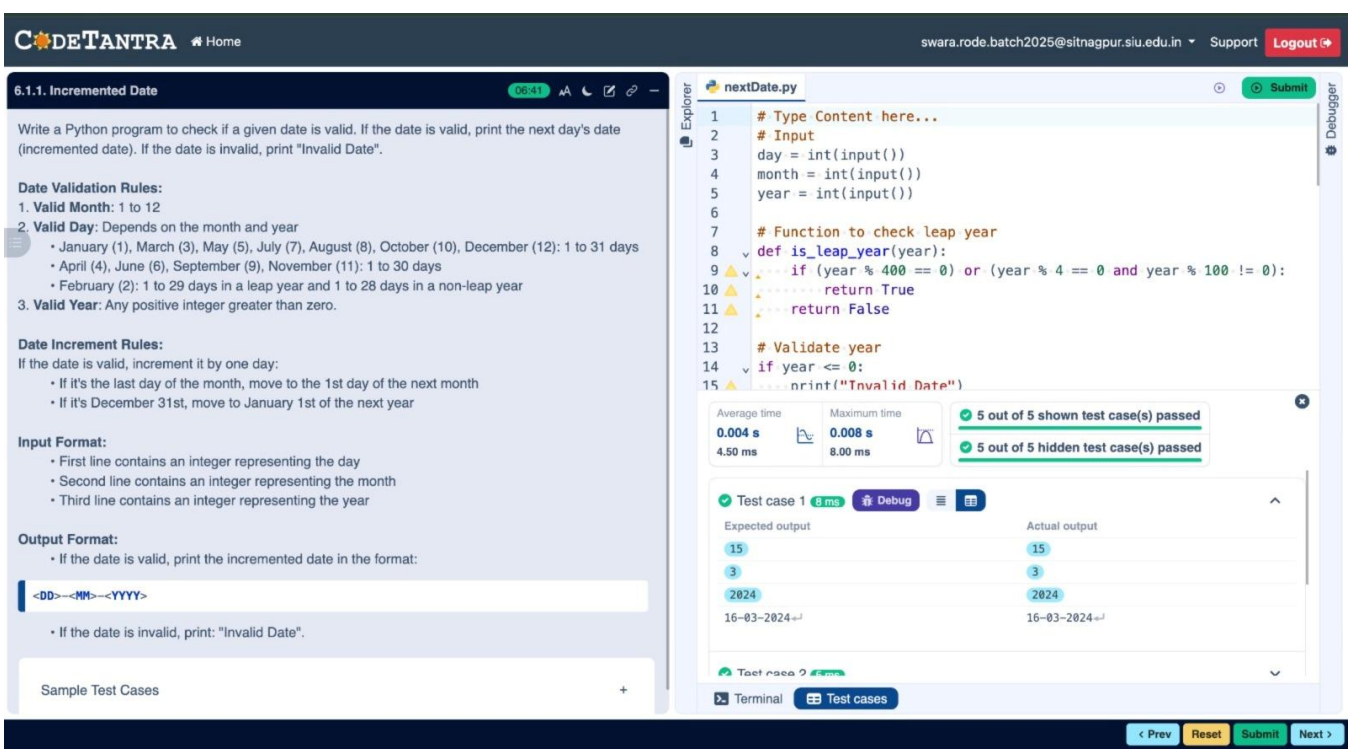


4. Python Program

```
d = int(input())
m = int(input())
y = int(input())

flag = True
maxdays = 0

# Check valid year and month bounds
ifi y <= 0 or m < 1 or m > 12:
   flag = False

# Determine maximum days in the given month ifi year and month are valid
ifi flag:
   ifi m in (1, 3, 5, 7, 8, 10, 12):
      maxdays = 31
   elifi m in (4, 6, 9, 11):
      maxdays = 30
   elifi m == 2:
      # Leap year check
      ifi (y % 400 == 0) or (y % 4 == 0 and y % 100 != 0):
         maxdays = 29
      else:
```

```
        maxdays = 28

    # Check ifi the day is within the valid range fior the month
    ifi d < 1 or d > maxdays:
        flag = False

# Increment date or print invalid message
ifi flag:
    d += 1
    # Check ifi we need to roll over to the next month
    ifi d > maxdays:
        d = 1
        m += 1
        # Check ifi we need to roll over to the next year
        ifi m > 12:
            m = 1
            y += 1

    # Print in zero-padded DD-MM-YYYY fiormat
    print(fi"{d:02d}-{m:02d}-{y:04d}")
else:
    print("Invalid Date")
```

5. Experiment Screenshot



Experiment - 6.1.2. Factorial ofi a Number

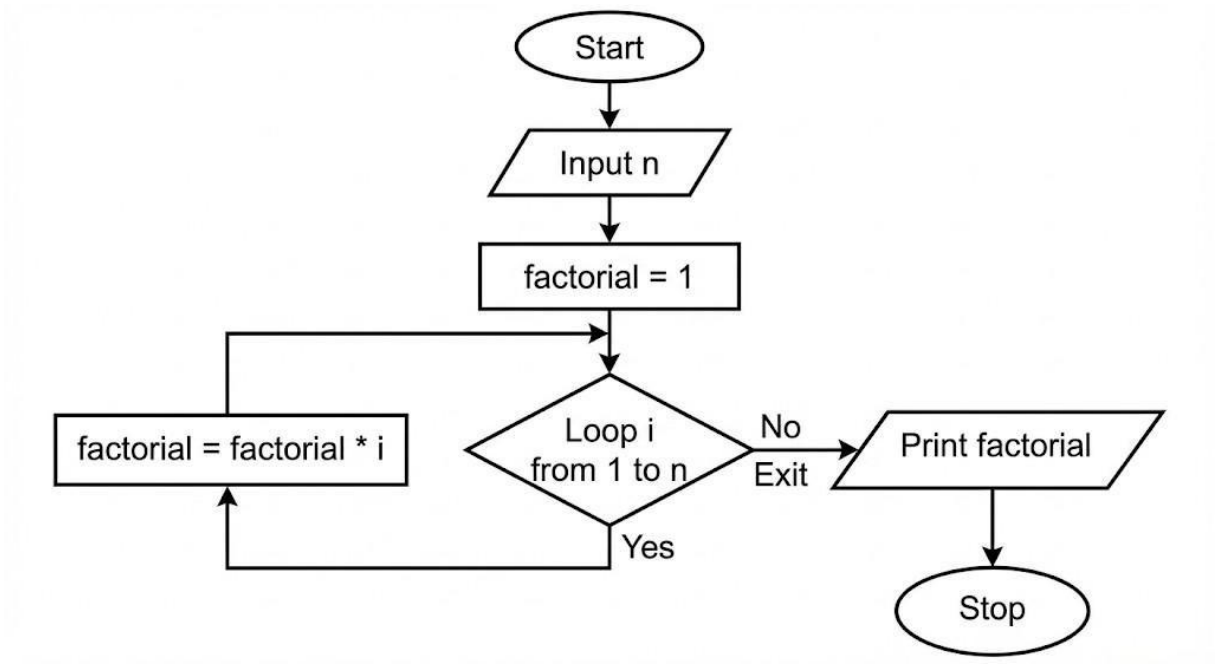
1. Aim

To design and implement a Python program that calculates the fiactorial ofi a given integer *n* using a fior loop. The program reads the integer, computes its fiactorial iteratively, and prints the final result.

2. Pseudocode

1. **START**
2. **READ** a single integer firom the user and **STORE** it in the variable n.
3. **INITIALIZE** a variable fiactorial to 1.
4. **FOR** each number i in the range firom 1 to n (inclusive):
    - **MULTIPLY** fiactorial by i (fiactorial = fiactorial * i).
    - **STORE** the new value back into fiactorial.
5. **END FOR** loop once all numbers up to n have been multiplied.
6. **PRINT** the final value ofi fiactorial.
7. **END**

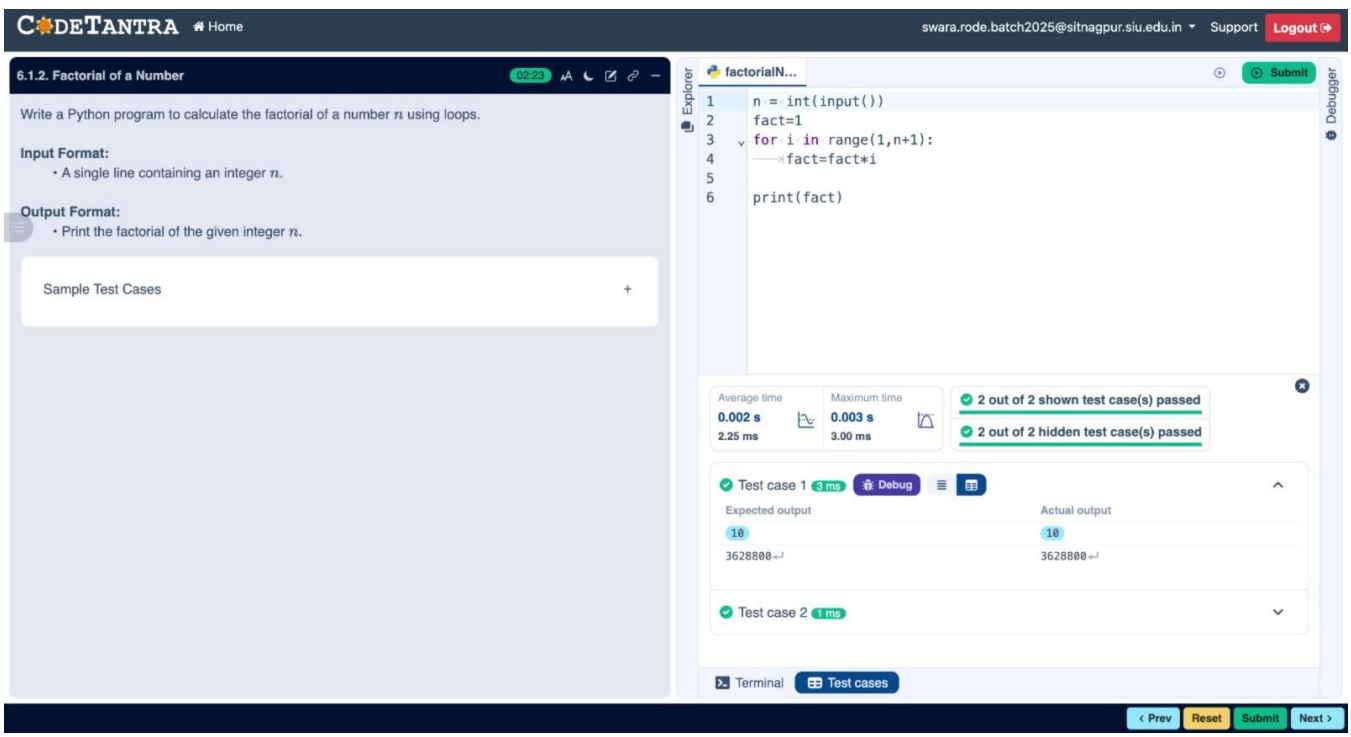
3. Flowchart



4. Python Program

```
n = int(input())

fiactorial = 1

fior i in range(1, n + 1):
   fiactorial *= i

print(fiactorial)
```

5. Experiment Screenshot



Experiment - 7.1.1. Inverted Star Pattern

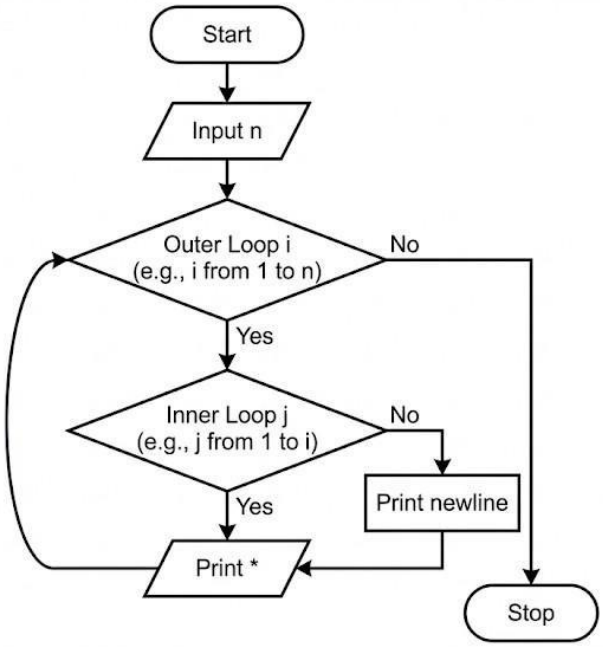
1. Aim

To design and implement a Python program that prints an inverted right-angled triangle star pattern. For a given integer *n*, the program uses nested loops to print *n* rows ofi stars, starting with *n* stars in the first row and decreasing by one star in each subsequent row until the last row has 1 star.

2. Pseudocode

1. **START**
2. **READ** a single integer firom the user and **STORE** it in the variable n.
3. **FOR** each number i in the range starting firom n down to 1 (decrementing by 1):
    - **FOR** each number j in the range firom 0 to i - 1:
        - **PRINT** a star "*" without a newline character at the end.
    - **END INNER FOR**
    - **PRINT** an empty line (newline character) to move to the next row.
4. **END OUTER FOR**
5. **END**

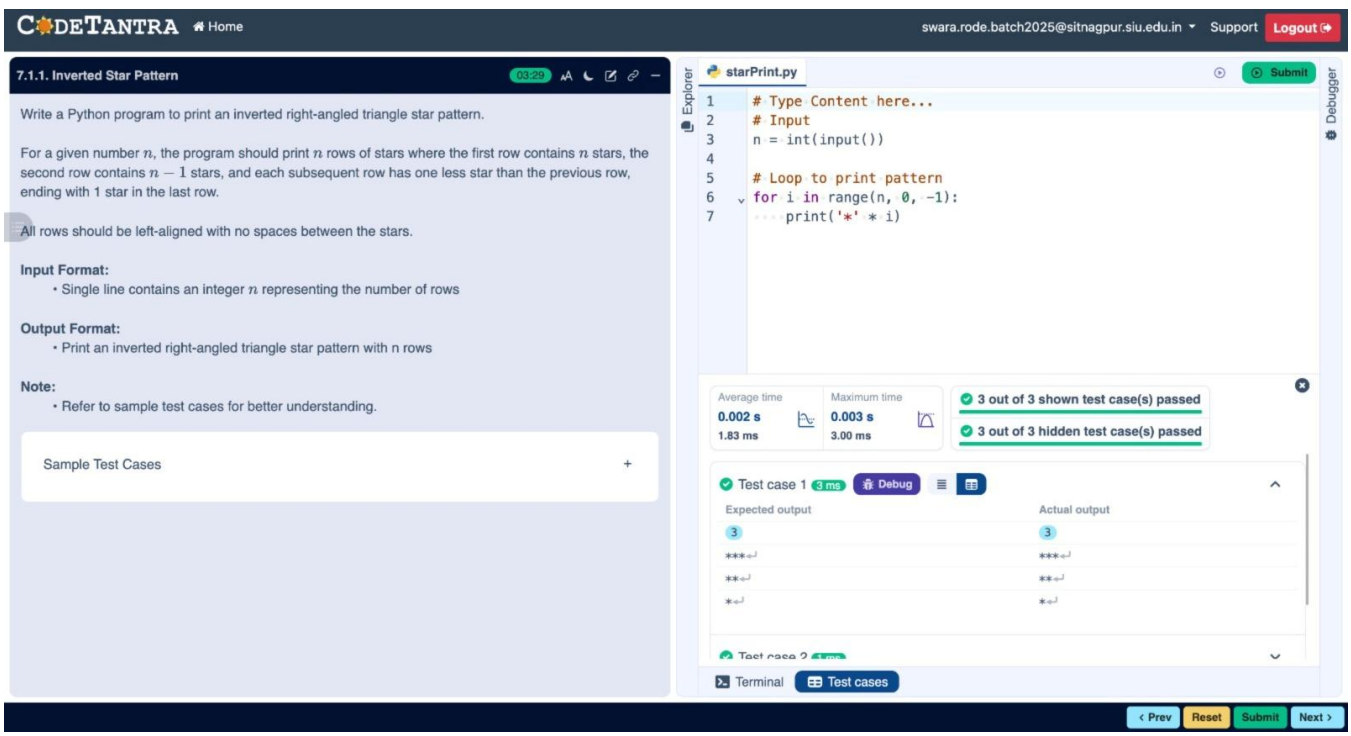
3. Flowchart



4. Python Program

```
n = int(input())

fior i in range(n, 0, -1):
   fior j in range(i):
      print("*", end="")
   print()
```

5. Experiment Screenshot



Experiment - 7.1.2. Possible Combinations firom Three Digits

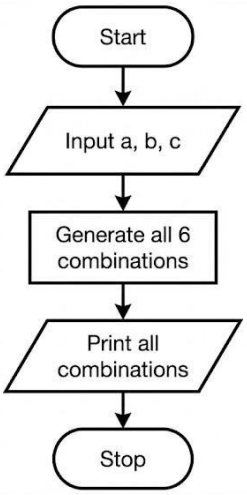
1. Aim

To design and implement a Python program that accepts three single-digit inputs (separated by spaces) and prints all six possible 3-digit combinations ofi these digits. The program must treat the inputs as strings to maintain any leading zeros through concatenation.

2. Pseudocode

1. **START**
2. **READ** a single line ofi input firom the user, split it by spaces, and **STORE** the three string values into variables a, b, and c.
3. **PRINT** the concatenation ofi a, b, and c (representing the combination abc).
4. **PRINT** the concatenation ofi a, c, and b (representing the combination acb).
5. **PRINT** the concatenation ofi b, a, and c (representing the combination bac).
6. **PRINT** the concatenation ofi b, c, and a (representing the combination bca).
7. **PRINT** the concatenation ofi c, a, and b (representing the combination cab).
8. **PRINT** the concatenation ofi c, b, and a (representing the combination cba).
9. **END**

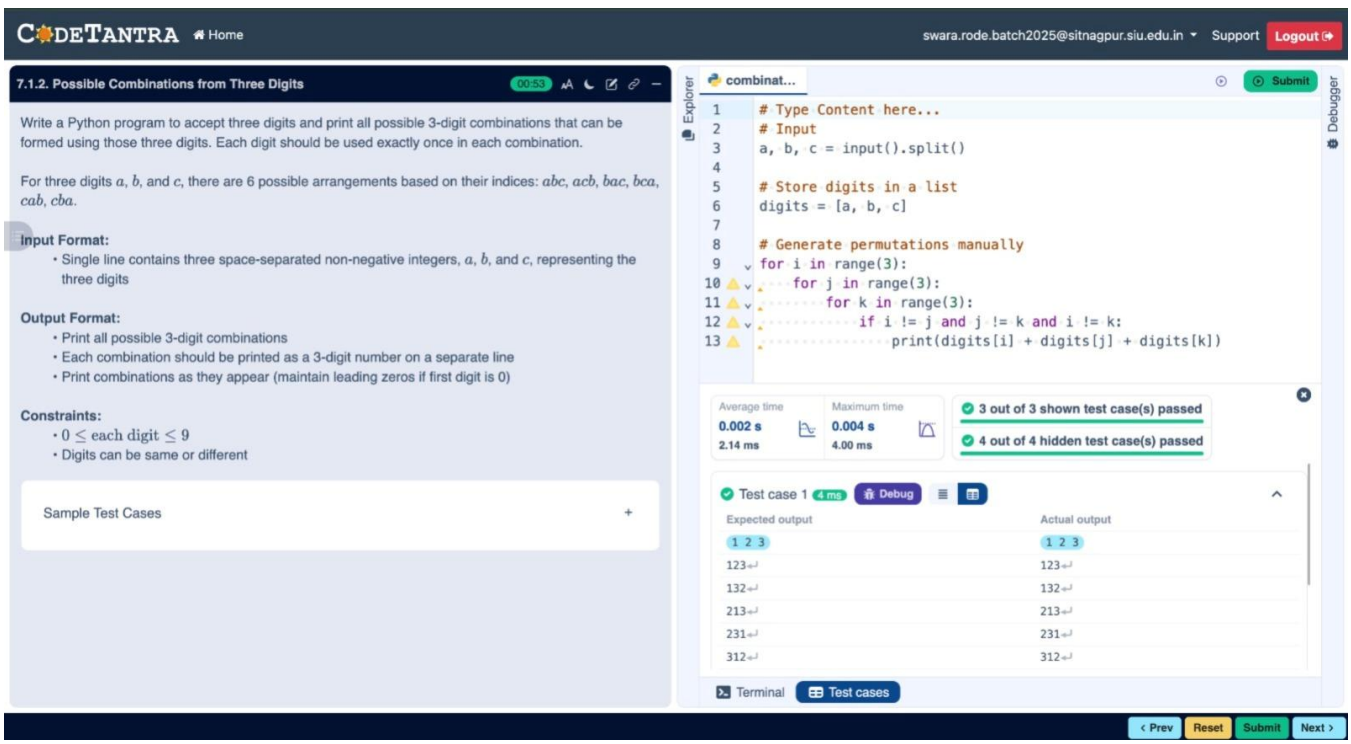
3. Flowchart



4. Python Program

```
a, b, c = input().split()

print(a + b + c)
print(a + c + b)
print(b + a + c)
print(b + c + a)
print(c + a + b)
print(c + b + a)
```

5. Experiment Screenshot



Experiment - 8.1.1. Multiplication ofi Two Square Matrices

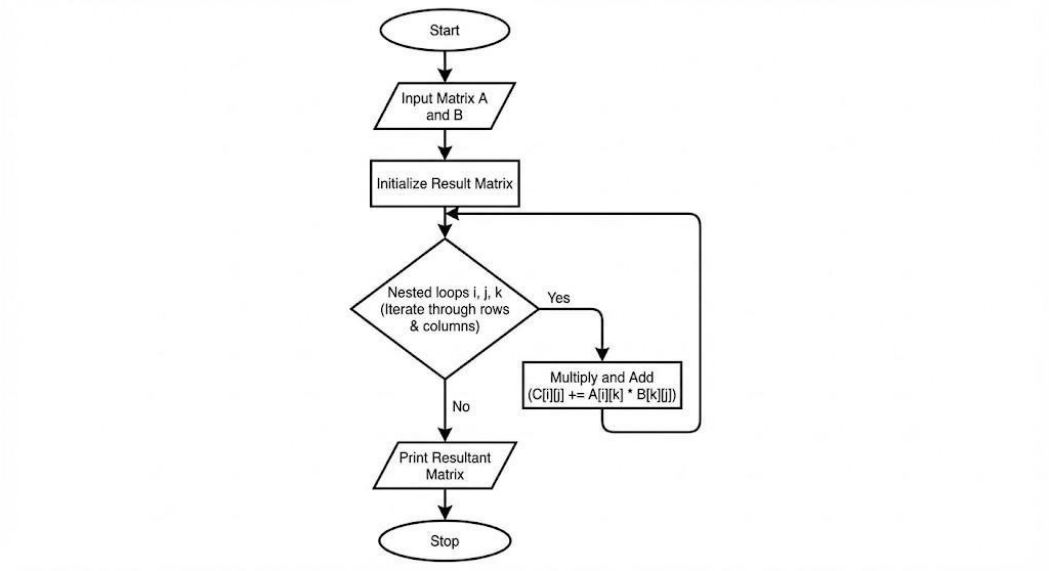
1. Aim

To design and implement a Python program that perfiorms the multiplication ofi two square matrices ofi dimension $n \times n$. The program takes the dimension and the elements ofi both matrices as input firom the user and computes their product.

2. Pseudocode

1. **START**
2. **PRINT** "dimension: " and **READ** an integer firom the user, storing it in the variable n.
3. **PRINT** "first matrix:" and **INITIALIZE** an empty list A.
4. **FOR** each row i firom 0 to n-1:
    - **READ** space-separated integers firom the user, convert them into a list, and append to A.
5. **PRINT** "second matrix:" and **INITIALIZE** an empty list B.
6. **FOR** each row i firom 0 to n-1:
    - **READ** space-separated integers firom the user, convert them into a list, and append to B.
7. **INITIALIZE** a result matrix ofi size $n \times n$ filled with zeros.
8. **FOR** i in range n (iterating through rows ofi A):
    - **FOR** j in range n (iterating through columns ofi B):
        - **FOR** k in range n (iterating through rows ofi B):
            - **ADD** the product ofi A[i][k] and B[k][j] to result[i][j].
9. **PRINT** "Resultant Matrix:".
10. **FOR** each row r in the result matrix:
    - **PRINT** the elements ofi r separated by spaces.
11. **END**

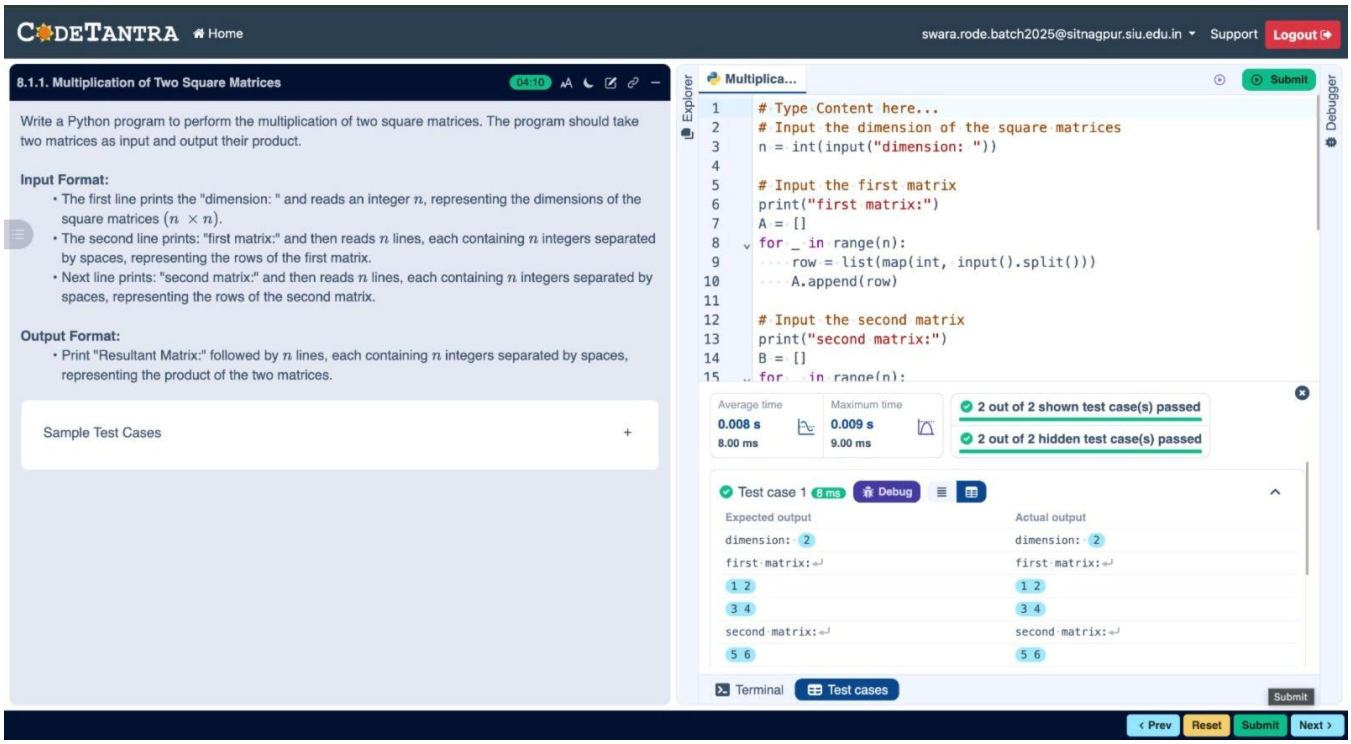
3. Flowchart



4. Python Program

```
n = int(input("dimension: "))

print("first matrix:")
A = []
fior i in range(n):
   A. append(list(map(int, input().split())))

print("second matrix:")
B = []
fior i in range(n):
   B. append(list(map(int, input().split())))

result = [[0]*n fior _ in range(n)]

# Matrix multiplication logic
fior i in range(n):
   fior j in range(n):
      fior k in range(n):
         result[i][j] += A[i][k] * B[k][j]

print("Resultant Matrix:")
fior r in result:
   print(*(r))
```

5. Experiment Screenshot



Experiment - 8.1.2. Prime Numbers in a Given Range

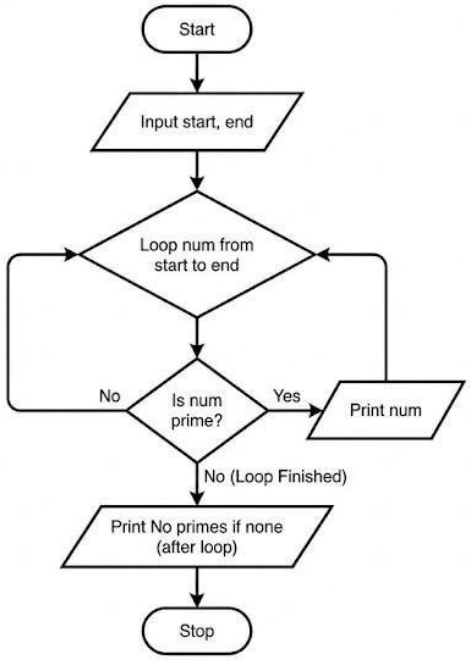
1. Aim

To design and implement a Python program that finds and prints all prime numbers within a specified range [start, end] (inclusive). The program determines the primality ofi each number and prints it. Ifi no prime numbers exist within the given range, the program prints "No primes".

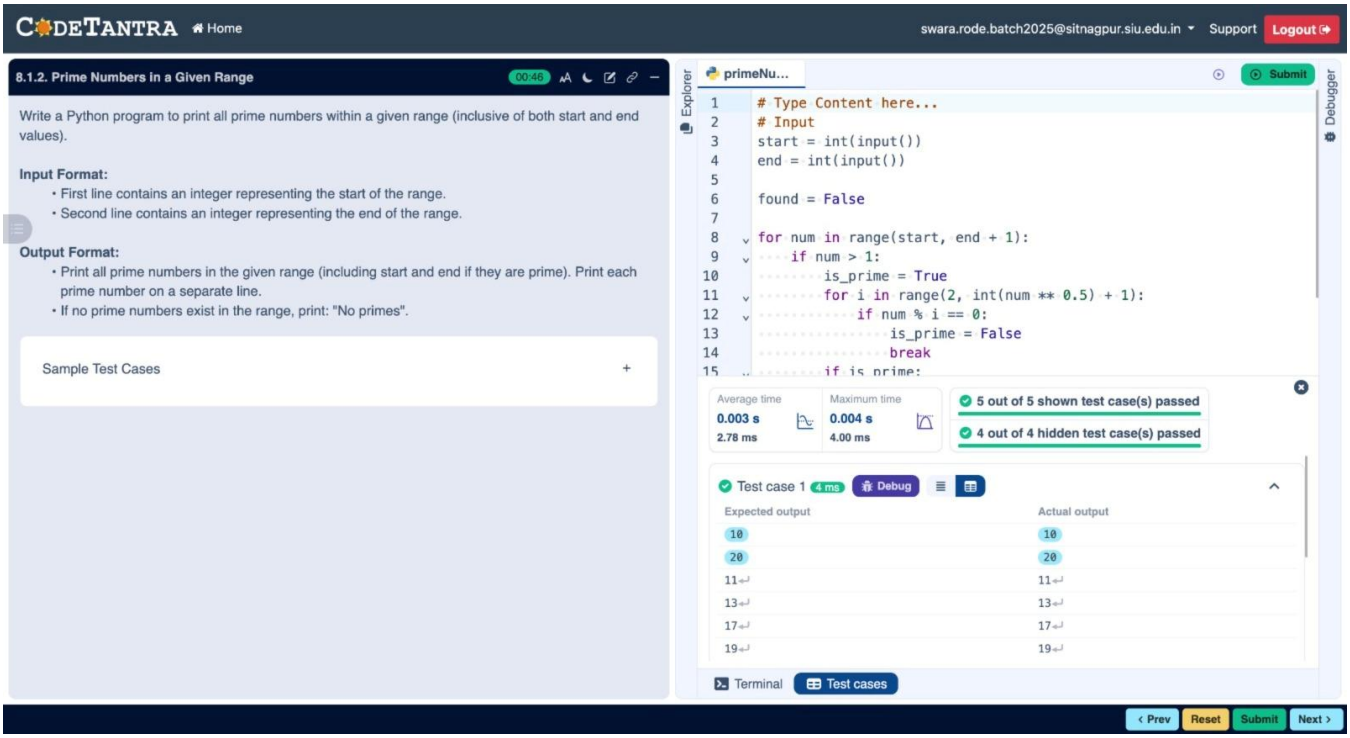
2. Pseudocode

1. **START**
2. **READ** an integer representing the start ofi the range and **STORE** it in start.
3. **READ** an integer representing the end ofi the range and **STORE** it in end.
4. **INITIALIZE** a boolean variable fiound to False to keep track ofi whether any primes exist.
5. **FOR** each number num in the range firom start to end (inclusive):
    - **IF** num > 1 (since 1 and numbers below it are not prime):
        - **INITIALIZE** is_prime to True.
        - **FOR** each number i firom 2 up to the integer square root ofi num (int(num**0.5) + 1):
            - **IF** num is divisible by i (num % i == 0):
                - **SET** is_prime to False.
                - **BREAK** the inner loop.
        - **IF** is_prime is True:
            - **PRINT** the prime number num.
            - **SET** fiound to True.
6. **IF** fiound is False (meaning no primes were printed):
    - **PRINT** "No primes".
7. **END**

3. Flowchart



4. Python Program

```
start = int(input())
end = int(input())

fiound = False

fior num in range(start, end + 1):
   ifi num > 1:
      is_prime = True
      fior i in range(2, int(num**0.5) + 1):
         ifi num % i == 0:
            is_prime = False
            break

      ifi is_prime:
         print(num)
         fiound = True

ifi not fiound:
   print("No primes")
```

5. Experiment Screenshot